

The Definitive Guide to Power Query M

MODULE

7

Control Flow and Advanced M

Master the M language itself: conditional logic, let...in structure, the each _ shorthand, error handling with try...otherwise, user-defined error messages, function definitions, and recursive functions with @. Write defensive, reusable, professional-grade M that handles real-world data.

Module 7: Control Flow and Advanced M

Module Contents

■ Module Overview

■ Lesson 1: The let...in Expression

- Structure and Evaluation Order
- Named Steps as Documentation
- Referencing Steps and Lazy Evaluation
- Nested let Expressions

■ Lesson 2: Conditional Logic — if/then/else

- Basic if/then/else
- Nested Conditionals and Chained else if
- Conditional in Table.AddColumn
- Conditional vs. Lookup — When to Use Each

■ Lesson 3: The each Keyword and Anonymous Functions

- What each _ Really Means
- each vs (x) => Syntax
- Functions as First-Class Values
- Passing Functions as Arguments

■ Lesson 4: Function Definitions

- Named Functions with let...in
- Type Annotations and Return Types
- Optional Parameters with ?? and record options
- Closures — Capturing Variables

■ Lesson 5: Error Handling — try...otherwise

- try...otherwise — Safe Evaluation
- try Expression — Structured Error Record
- error — Raising User Errors
- Error.Record and Structured Errors
- Defensive Patterns for Production ETL

■ Lesson 6: Recursive Functions with @

- The @ Self-Reference Operator
- Factorial and Fibonacci — Classic Recursion
- Practical Recursion: Hierarchical Data Flattening
- Recursion vs List.Generate — Choosing the Right Tool

■ Lesson 7: Advanced Language Patterns for PL-300

- Pattern 1: Reusable ETL Function Library
- Pattern 2: Multi-Step Validation Pipeline
- Pattern 3: Dynamic Dispatch with Function Records
- Pattern 4: Memoisation with Table.Buffer
- Pattern 5: Type-Safe Generic Functions

■ Demonstration 1: Building a Reusable ETL Function Library

■ Demonstration 2: Defensive Error-Handling Pipeline

■ Lab 7: Control Flow and Advanced M

- Exercise 1: let...in and Conditional Logic
- Exercise 2: Function Definitions and Closures
- Exercise 3: Error Handling and Defensive ETL
- Exercise 4: Recursive Functions and Advanced Patterns

■ Module Review and Takeaways

Module Overview

The first six modules taught you what M can do. This module teaches you how M works — the language mechanics that determine when expressions are evaluated, how functions are defined, how errors propagate, and how recursive structures are expressed. This knowledge is what separates a developer who uses M from one who masters it.

Control flow in M is fundamentally different from imperative languages. M has no loops, no mutable variables, and no statements — only expressions. Every piece of M code is an expression that evaluates to a value. `let...in` is not a sequence of statements; it is a set of simultaneous name bindings. `if/then/else` is not a conditional statement; it is a conditional expression that always produces a value. Understanding this distinction is the key to writing correct, efficient, and composable M code.

NOTE

This module is the capstone of the language fundamentals series. Modules 1–6 are prerequisites. After this module, Modules 8–12 apply all language knowledge to API connectors, custom connectors, query folding, performance optimisation, and real-world Power BI architecture patterns.

Before You Begin

- Complete Modules 1–6 and all lab exercises.
- Have a Power BI Desktop project with the Contoso model from previous labs.
- Download lab files: `ValidationRules.xlsx` and `HierarchyData.json`.

LESSON 1

The let...in Expression

After completing this lesson, you will be able to:

- Explain that let...in defines simultaneous name bindings, not sequential statements.
- Use named steps as self-documenting intermediate values.
- Reference steps by name and understand lazy evaluation.
- Write nested let expressions for scoped intermediate values.

Structure and Evaluation Order

The let...in expression is the fundamental structure of every M query. Its syntax is deceptively simple but its semantics are important to understand correctly:

```
// Syntax of let...in
let
Name1 = expression1,
Name2 = expression2, // may reference Name1
Name3 = expression3 // no comma on last binding
in
Name3 // the 'in' clause is the result of the entire expression

// Simple example
let
Base = 100,
Rate = 0.15,
Tax = Base * Rate, // references Base and Rate
Total = Base + Tax // references Base and Tax
in
Total // = 115
```

WARNING

let...in bindings are NOT sequential statements. They are simultaneous name bindings in a mathematical sense. M evaluates only the bindings actually needed to produce the 'in' result — this is called lazy evaluation. A binding that is never referenced is never evaluated. This has important implications for performance and for error handling.

Named Steps as Documentation

In Power Query, each named binding in a let...in expression becomes a step in the Applied Steps pane. Naming steps descriptively is therefore both good M practice and directly improves the experience for anyone maintaining the query:

```
// Poor naming – Applied Steps shows: Source, Custom1, Filtered Rows, Custom2
let
Source = Csv.Document(...),
Custom1 = Table.PromoteHeaders(Source),
#"Filtered Rows" = Table.SelectRows(Custom1, each [Amount] > 0),
Custom2 = Table.AddColumn(#"Filtered Rows", "Tax", each [Amount]*0.1)
in
Custom2

// Good naming – self-documenting query
```

```
let
RawCSV = Csv.Document(File.Contents("Sales.csv"), [Delimiter=","]),
WithHeaders = Table.PromoteHeaders(RawCSV),
ValidRows = Table.SelectRows(WithHeaders, each [Amount] > 0),
WithTax = Table.AddColumn(ValidRows, "Tax", each [Amount]*0.1, type number),
TypedResult = Table.TransformColumnTypes(WithTax, {"Date",type date},{"Amount",type number})
in
TypedResult
// Each step name explains what the step produces, not what function was used
```

Nested let Expressions

A `let...in` can be nested inside any other expression — inside an if branch, inside a function body, inside another let binding. This is used to create local variables scoped to a single computation:

```
// Nested let inside a Table.AddColumn function
Table.AddColumn( Orders, "ShippingCategory",
each
let
Weight = [Weight_kg],
Zone = [DestZone],
BaseRate = if Zone = "EU" then 5.0 else 12.0,
FinalRate= BaseRate * (if Weight > 10 then 1.5 else 1.0)
in
if FinalRate > 15 then "Express" else "Standard"
, type text )

// Nested let inside a conditional
Result =
let
IsWeekend = Date.DayOfWeek(Today, Day.Monday) >= 5
in
if IsWeekend then "Weekend rate" else "Weekday rate"
```

LESSON 2

Conditional Logic — if / then / else

After completing this lesson, you will be able to:

- Write single and chained if/then/else conditional expressions.
- Embed conditionals inside Table.AddColumn and Table.SelectRows.
- Decide when to use conditional logic vs a lookup record.
- Understand that M if/then/else is an expression, not a statement.

Syntax	if then else	
Description	A conditional expression. Always produces a value — both branches must be present. Unlike SQL CASE WHEN, there is no ELSE equivalent to null — you must write 'else null' explicitly.	
Parameter	Type	Description
condition	logical	Any expression that evaluates to true or false.
trueValue	any	Returned when condition is true.
falseValue	any	Returned when condition is false. Required.
Returns	The value of trueValue or falseValue depending on condition.	
Example 1	if [Score] >= 50 then "Pass" else "Fail"	
Result 1	"Pass" or "Fail"	
Example 2	if [Amount] = null then 0 else [Amount]	
Result 2	0 for nulls, otherwise Amount	

NOTE

There is no 'else if' keyword — chain conditions by nesting another if in the else branch: if A then X else if B then Y else Z.

Chained conditions (the M equivalent of SWITCH / CASE WHEN):

```
// Chained if / else if / else – equivalent to SQL CASE WHEN
Table.AddColumn( Sales, "SalesGrade",
each
if [Revenue] >= 100000 then "Platinum"
else if [Revenue] >= 50000 then "Gold"
else if [Revenue] >= 10000 then "Silver"
else "Bronze"
, type text )

// Compound conditions with 'and' / 'or'
Table.SelectRows( Sales,
each [Date] >= #date(2024,1,1)
and [Region] <> null
and ( [Revenue] > 0 or [ReturnFlag] = true ) )

// Null-safe conditional (never apply arithmetic to a null)
Table.AddColumn( Orders, "Margin",
```

```
each if [Revenue] = null or [Cost] = null  
then null  
else [Revenue] - [Cost]  
, type nullable number )
```

TIP

When you have more than 3-4 conditions mapping values to outputs, a lookup record is cleaner than chained if/else. `Record.FieldOrDefault(Lookup, [Key], "Unknown")` replaces a long if/else chain with a single expression and is trivial to extend. Use if/else for logic; use records for data-driven mapping.

LESSON 3

The each Keyword and Anonymous Functions

After completing this lesson, you will be able to:

- Explain that each is shorthand for a single-parameter anonymous function.
- Distinguish when to use each _ vs explicit (x) => syntax.
- Pass functions as arguments to higher-order functions.
- Understand that M functions are first-class values.

Syntax	each // equivalent to: (_) =>	
Description	Creates an anonymous single-parameter function. The underscore _ is the implicit parameter. In table functions, _ is the current row record. In list functions, _ is the current list element. The each keyword exists purely for readability.	
Parameter	Type	Description
_	any	The implicit parameter. In table contexts: the current row as a record. In list contexts: the current list element.
Returns	A function value: (_) => expression	
Example 1	each [Amount] * 0.1	
Result 1	(_) => _[Amount] * 0.1 (equivalent)	
Example 2	each _ > 100	
Result 2	(_) => _ > 100 (equivalent)	

NOTE

each _ works only for single-parameter functions. For two-parameter functions (e.g. List.Accumulate's accumulator), use explicit syntax: (state, current) => expression.

each vs explicit function syntax — when to use each:

```
// each is ideal for single-parameter row/element functions
Table.AddColumn( Sales, "Tax", each [Amount] * 0.1 )
Table.SelectRows( Sales, each [Amount] > 1000 )
List.Transform( {1,2,3,4}, each _ * 2 )
List.Select( {1,2,3,4,5,6}, each _ > 3 )

// Use explicit (x) => when you need:
// 1. Two parameters (accumulator functions)
List.Accumulate({1,2,3,4,5}, 0, (state, current) => state + current)

// 2. A named parameter for readability in complex logic
List.Transform( Products,
(product) =>
let
Price = product[UnitPrice],
Discount = if product[VIP] then 0.2 else 0.0
in
Price * (1 - Discount)
)
```

```
)  
  
// 3. Recursive functions (the parameter must have a name to self-reference)  
Factorial = (n as number) as number =>  
if n <= 1 then 1 else n * @Factorial(n - 1)
```

Functions as First-Class Values

In M, functions are values just like numbers or text. They can be stored in variables, passed as arguments, returned from other functions, and stored in lists and records:

```
// Store a function in a variable  
Double = (x) => x * 2  
Result = Double(5) // = 10  
  
// Pass a function as an argument  
ApplyTwice = (fn as function, x as number) as number => fn(fn(x))  
ApplyTwice(Double, 3) // = 12 (Double(Double(3)) = Double(6) = 12)  
  
// Store functions in a record – the dispatch table pattern (see Lesson 7)  
Aggregators = [  
Sum = List.Sum,  
Avg = List.Average,  
Max = List.Max,  
Min = List.Min  
],  
// Select the aggregator at runtime  
Method = "Avg",  
Result = Record.Field(Aggregators, Method)( {10,20,30,40} )  
// = List.Average({10,20,30,40}) = 25
```

LESSON 4

Function Definitions

After completing this lesson, you will be able to:

- Define named functions with parameter and return type annotations.
- Implement optional parameters using nullable types and the ?? operator.
- Apply the defaults-and-overrides pattern with a record options parameter.
- Understand closures — functions that capture variables from their enclosing scope.

Syntax	(param1 as type, param2 as type, ...) as returnType => expression	
Description	Defines a function with typed parameters and an optional return type annotation. Type annotations are optional but strongly recommended for functions shared across queries or used in Power BI Dataflows.	
Parameter	Type	Description
param	any	Named parameter with optional type annotation.
returnType	type	Optional return type after 'as'.
expression	any	The function body — any M expression that uses the parameters.
Returns	A function value that can be called with matching arguments.	
Example 1	(n as number) as number => n * 2	
Result 1	A typed doubling function	
Example 2	(a as number, b as number) => if a > b then a else b	
Result 2	Max of two numbers	

NOTE

Functions defined in a Query become available to all other queries in the same Power BI file. Create a blank query named after the function; its result IS the function.

Optional parameters and the defaults-and-overrides pattern:

```
// Option 1: nullable type makes parameter optional – caller passes null to use default
FormatCurrency = (amount as number, optional decimals as nullable number) as text =>
let
    D = decimals ?? 2 // ?? = null-coalescing: use right side if left is null
in
    Text.From(Number.Round(amount, D)) & " USD"

FormatCurrency(1234.5) // = "1234.5 USD" (uses default 2)
FormatCurrency(1234.5, 0) // = "1235 USD"

// Option 2: record options – more extensible, supports many optional parameters
TransformTable = (source as table, optional opts as nullable record) as table =>
let
    Defaults = [ RoundTo=2, RemoveNulls=true, MaxRows=null, Currency="USD" ],
    Options = Record.Combine({ Defaults, opts ?? [] }),
    Cleaned = if Options[RemoveNulls]
```

```
then Table.SelectRows(source, each [Amount] <> null)
else source,
Rounded = Table.TransformColumns(Cleaned, {"Amount",
each Number.Round(_, Options[RoundTo]), type number})),
Limited = if Options[MaxRows] = null then Rounded
else Table.FirstN(Rounded, Options[MaxRows])
in
Limited

// Caller only specifies what differs from defaults:
TransformTable(RawSales, [ MaxRows = 500, Currency = "GBP" ])
```

Closures — Capturing Variables

A function defined inside a `let...in` expression automatically captures variables from the enclosing scope. This is called a closure:

```
// The inner function captures 'Threshold' from the outer let binding
let
Threshold = 5000,
IsHighValue = (amount as number) as logical => amount >= Threshold, // closure!
Sales = {1200, 6500, 3400, 8900, 500},
HighSales = List.Select(Sales, IsHighValue) // passes the closure as a function
in
HighSales // = {6500, 8900}

// Closures enable factory functions
MakeMultiplier = (factor as number) as function => (x as number) => x * factor
Triple = MakeMultiplier(3)
Tenfold = MakeMultiplier(10)
Triple(7) // = 21
Tenfold(7) // = 70
```

LESSON 5

Error Handling — try ... otherwise

After completing this lesson, you will be able to:

- Use try...otherwise to safely evaluate expressions that may fail.
- Inspect the full error record returned by try without otherwise.
- Raise user-defined errors with error and Error.Record.
- Build defensive ETL patterns that isolate, log, and recover from errors.

Syntax	try otherwise	
Description	Attempts to evaluate expression. If it succeeds, returns the result. If it raises any error, silently discards the error and returns fallbackValue instead. This is the M equivalent of try/catch in most programming languages.	
Parameter	Type	Description
expression	any	The expression to attempt. May be any M expression.
fallbackValue	any	Returned if expression raises an error.
Returns	The result of expression on success, or fallbackValue on any error.	
Example 1	try Number.FromText("abc") otherwise 0	
Result 1	0 (text is not a valid number)	
Example 2	try Date.From([DateCol]) otherwise null	
Result 2	null if DateCol contains invalid dates	

NOTE

try...otherwise catches ALL errors — there is no way to catch specific error types. If you need to inspect what went wrong, use 'try expression' without 'otherwise' to access the full error record.

Syntax	try	
Description	Evaluates expression and returns a record with two fields: HasError (logical) and Value (the result) or Error (the error record). This allows you to inspect error details, log them, and branch on them.	
Parameter	Type	Description
expression	any	The expression to attempt.
Returns	A record: [HasError=false, Value=result] on success, or [HasError=true, Error=[Reason=..., Message=..., Detail=...]] on failure.	
Example 1	try Number.FromText("abc")	
Result 1	[HasError=true, Error=[Reason="Expression.Error", ...]]	
Example 2	try Number.FromText("42")	
Result 2	[HasError=false, Value=42]	

NOTE

Use (try expr)[HasError] to get a logical. Use (try expr)[Value] for the result when HasError=false. Use (try expr)[Error][Message] to extract the error message.

Production error handling patterns:

```
// Pattern 1: Safe type conversion with fallback
Table.AddColumn( Source, "ParsedDate",
each try Date.From([RawDate]) otherwise null, type nullable date )

// Pattern 2: Inspect and log the error
Table.AddColumn( Source, "ParseResult",
each
let attempt = try Number.FromText([Value])
in if attempt[HasError]
then [ Status="Error", Detail=attempt[Error][Message], Parsed=null ]
else [ Status="OK", Detail=null, Parsed=attempt[Value] ]
, type record )

// Pattern 3: Validate before applying, raise a descriptive error if invalid
ValidateAndLoad = (source as table) as table =>
let
Required = {"CustomerID","Date","Revenue"},
Missing = List.Difference(Required, Table.ColumnNames(source))
in
if not List.IsEmpty(Missing)
then error Error.Record(
"Schema.Error",
"Required columns missing: " & Text.Combine(Missing, ", "),
[ RequiredColumns = Required, SourceColumns = Table.ColumnNames(source) ]
)
else source

// Pattern 4: Wrap an entire query in try for graceful degradation
SafeResult = try FullQuery otherwise
#table({"Status","Message"}, {{ "Error","Query failed – check source connectivity" }})
```

Syntax	<code>error</code> or <code>error Error.Record(reason, message, optional detail)</code>	
Description	Raises a user-defined error that immediately stops evaluation of the current expression and propagates the error to the caller (or to an enclosing try).	
Parameter	Type	Description
<code>message</code>	text or record	A text message, or an <code>Error.Record</code> for structured errors.
<code>reason</code>	text	Short error category: e.g. "Schema.Error", "Validation.Failed".
<code>detail</code>	record	Optional. Any additional diagnostic information.
Returns	Never returns — raises an error that propagates upward.	
Example 1	<code>error "Age cannot be negative"</code>	
Result 1	Raises error with text message	
Example 2	<code>error Error.Record("Validation","Null key",null)</code>	
Result 2	Raises structured error	

NOTE

Always use `Error.Record` for production errors — it produces a structured error record with Reason, Message, and Detail fields that are visible in Power BI's error dialogs and make debugging far easier than plain text messages.

LESSON 6

Recursive Functions with @

After completing this lesson, you will be able to:

- Use the @ operator to enable a function to call itself.
- Implement factorial and Fibonacci as canonical recursion examples.
- Flatten hierarchical (tree-structured) data with recursive M.
- Choose between recursion and List.Generate for iterative patterns.

Syntax	let FuncName = (params) => ... @FuncName(nextArgs) ... in FuncName	
Description	The @ operator inside a let binding refers to the binding itself, enabling self-reference. Without @, a function cannot call itself because the name is not in scope within its own definition. @ makes recursion explicit and safe.	
Parameter	Type	Description
@FuncName	function	Reference to the enclosing named binding, enabling self-call.
Returns	Enables a function to call itself recursively.	
Example 1	let F = (n) => if n<=1 then 1 else n * @F(n-1) in F(5)	
Result 1	120 (5! = 120)	
Example 2	let F = (n) => if n<=1 then 1 else n * @F(n-1) in F(10)	
Result 2	3628800 (10!)	

NOTE

M does not have tail-call optimisation. Deep recursion (hundreds of levels) may cause stack overflow errors. For deep iteration, prefer List.Generate.

Classic recursion examples:

```
// Factorial
let
    Factorial = (n as number) as number =>
        if n <= 1 then 1 else n * @Factorial(n - 1)
in
    Factorial(6) // = 720

// Fibonacci
let
    Fib = (n as number) as number =>
        if n <= 1 then n else @Fib(n-1) + @Fib(n-2)
in
    Fib(10) // = 55
// Note: Fib above is exponentially slow. For large n, use List.Generate (Module 4).

// Sum a list recursively (illustrative – use List.Sum in practice)
let
    SumList = (lst as list) as number =>
```

```
if List.IsEmpty(lst) then 0
else List.First(lst) + @SumList(List.Skip(lst, 1))
in
SumList({1,2,3,4,5}) // = 15
```

Practical Recursion: Hierarchical Data Flattening

Recursion's real power in Power Query is traversing tree-structured data — common in organisational hierarchies, bill of materials, and nested JSON:

```
// Flatten an employee hierarchy to (EmployeeID, ManagerID, Level, Path)
// Hierarchy table: [EmployeeID, Name, ManagerID]

let
Hierarchy = HierarchyTable,

// Recursive function: find all ancestors of an employee
GetPath = (empID as text, depth as number) as text =>
let
managerRow = Table.SelectRows(Hierarchy,
each [EmployeeID] = empID),
managerID = if Table.IsEmpty(managerRow) then null
else managerRow[ManagerID]{0}
in
if managerID = null or depth > 10 // safety depth limit
then empID
else @GetPath(managerID, depth+1) & " > " & empID,

// Add Path and Level columns
WithPath = Table.AddColumn(Hierarchy, "AncestorPath",
each @GetPath([EmployeeID], 0), type text)
in
WithPath
```

WARNING

Always include a depth limit in recursive functions: if depth > 100 then error "Max depth exceeded". Without it, circular references or unexpected deep hierarchies cause a stack overflow that crashes the entire refresh. The depth limit converts an infinite loop into a recoverable error.

LESSON 7

Advanced Language Patterns for PL-300

After completing this lesson, you will be able to:

- Build and share a reusable ETL function library across queries.
- Implement a multi-step validation pipeline with structured error reporting.
- Apply dynamic dispatch using a function record.
- Write type-safe generic functions using Value.Is and type checks.

Pattern 1: Reusable ETL Function Library

```
// Create a blank query named 'ETL' – its result is a record of functions
// All other queries reference ETL[FunctionName](args)

let
    ETL = [
        // Rename and type a table from a config record
        StandardiseTable = (tbl as table, ColMap as record, TypeMap as record) as table =>
            let
                Renamed = Table.RenameColumns(tbl,
                    List.Zip({Record.FieldNames(ColMap), Record.FieldValues(ColMap)}),
                    MissingField.Ignore),
                Typed = Table.TransformColumnTypes(Renamed,
                    List.Zip({Record.FieldNames(TypeMap), Record.FieldValues(TypeMap)}))
            in Typed,

        // Safe date parser – returns null on invalid
        SafeDate = (v as any) as nullable date => try Date.From(v) otherwise null,

        // Filter to current year
        CurrentYear = (tbl as table, dateCol as text) as table =>
            let yr = Date.Year(DateTime.Date(DateTime.LocalNow()))
            in Table.SelectRows(tbl, each Date.Year(Record.Field(_, dateCol)) = yr),

        // Validate schema and error with detail
        ValidateSchema = (tbl as table, required as list) as table =>
            let missing = List.Difference(required, Table.ColumnNames(tbl))
            in if List.IsEmpty(missing) then tbl
            else error Error.Record("Schema", "Missing: " & Text.Combine(missing, ", "), null)
    ]
in
    ETL

// Usage in any other query:
// ETL[StandardiseTable](RawData, ColMap, TypeMap)
// ETL[SafeDate]("2024-13-45") → null (not an error)
// ETL[CurrentYear](Sales, "Date")
```

Pattern 2: Multi-Step Validation Pipeline

```
// Run a table through a list of validation checks; collect all failures
let
    Source = RawSalesData,
```

```
// Each check is a record: [Name, Test, Message]
Checks = {
[ Name = "HasRows", Test = Table.RowCount(Source) > 0,
Message = "Source table is empty" ],
[ Name = "NotNullKeys", Test = Table.RowCount(Table.SelectRows(Source,
each [CustomerID] = null)) = 0,
Message = "Null CustomerID values found" ],
[ Name = "ValidDates", Test = List.AllTrue(List.Transform(Source[Date],
each _ <> null and _ >= #date(2020,1,1))),
Message = "Dates outside valid range" ],
[ Name = "PositiveRevenue", Test = List.AllTrue(List.Transform(Source[Revenue],
each _ <> null and _ >= 0)),
Message = "Negative or null Revenue values" ]
},

// Run all checks; collect failures
Failures = List.Select(Checks, each not [Test]),
Messages = List.Transform(Failures, each [Message]),
AllPassed = List.IsEmpty(Failures)
in
if AllPassed
then Source
else error Error.Record(
"Validation.Failed",
Text.Combine(Messages, " | "),
[ FailedChecks = List.Transform(Failures, each [Name]) ]
)
```

Pattern 3: Dynamic Dispatch with Function Records

```
// Select and call a function by name at runtime
// No if/else chains needed – 0(1) lookup via record field access

let
Formatters = [
Currency = (n) => "$" & Text.From(Number.Round(n,2)),
Percent = (n) => Text.From(Number.Round(n*100,1)) & "%",
Integer = (n) => Text.From(Number.RoundDown(n,0)),
Default = (n) => Text.From(n)
],

// Runtime format selection – could come from a config record or table column
FormatType = "Currency",
ApplyFormat = Record.FieldOrDefault(Formatters, FormatType, Formatters[Default]),

Result = Table.AddColumn( Sales, "FormattedRevenue",
each ApplyFormat([Revenue]), type text )
in
Result
```

Pattern 4: Type-Safe Generic Functions

```
// A function that behaves differently based on input type
// Value.Is checks the type at runtime

SmartFormat = (v as any) as text =>
```

```
if v = null then "(null)"
else if Value.Is(v, type number) then Number.ToText(v)
else if Value.Is(v, type date) then Date.ToText(v, "yyyy-MM-dd")
else if Value.Is(v, type logical) then (if v then "Yes" else "No")
else if Value.Is(v, type list) then "[" & Text.Combine(
List.Transform(v, each SmartFormat(_)),",") & "]"
else Text.From(v)

// Apply to any column regardless of type
Table.TransformColumns( MixedTable,
List.Transform(Table.ColumnNames(MixedTable),
(col) => { col, SmartFormat, type text } ) )
```

DEMONSTRATION 1

Building a Reusable ETL Function Library

Scenario: Contoso's Power BI team has six nearly identical ETL queries, each with copy-pasted transformation logic. A change to one requires updating all six. The goal is to extract shared logic into a single ETL function library query.

Demonstration Steps

1	Open the existing Power BI file. Show the six source queries. Highlight the duplicated RenameColumns, TransformColumnTypes, FillDown, and SelectRows logic in each.
2	Create a new blank query named 'ETL'. In the Advanced Editor, define a record whose fields are functions: StandardiseTable, SafeDate, CurrentYear, ValidateSchema.
3	Replace the duplicated RenameColumns step in QueryA with ETL[StandardiseTable](...). Verify the output is identical.
4	Replace all six queries' shared steps with ETL function calls. Show that each query is now 6–8 steps instead of 20+.
5	Change the default rounding precision inside ETL[StandardiseTable]. Refresh all six queries — all pick up the change automatically.
6	Demonstrate ETL[ValidateSchema] raising a structured error with the missing column list. Show how the error appears in Power BI's error dialog with the detail record.
7	Discuss: this pattern is the M equivalent of a shared library or module system. It is the correct approach for enterprise Power BI models with many similar sources.

RESULT

A single ETL query containing all shared transformation functions. All six source queries are simplified by 60–70%. A schema change in any source produces a clear, structured error message. Updating shared logic requires editing one query.

DEMONSTRATION 2

Defensive Error-Handling Pipeline

Scenario: A Power BI model refreshes nightly from a third-party API. The API occasionally returns malformed JSON, missing fields, or invalid data types. A single bad record currently crashes the entire refresh. The goal is to build a row-level error isolation pattern.

Demonstration Steps

1	Show the current query failing on a single bad row. Trace the error to an invalid date in one record crashing <code>Table.TransformColumnTypes</code> .
2	Replace <code>Table.TransformColumnTypes</code> for the Date column with a <code>Table.AddColumn</code> using <code>try Date.From([RawDate])</code> otherwise null. Show that bad rows now produce null, not errors.
3	Extend to full row-level error capture: use 'try (row-level expression)' without otherwise to get the error record. Add an <code>ErrorMessage</code> column with the error detail.
4	Add a validation step that counts error rows: <code>ErrorCount = Table.RowCount(Table.SelectRows(WithErrors, each [ErrorMessage] <> null))</code> .
5	Conditionally raise a query-level error if too many rows failed: <code>if ErrorCount > 100 then error Error.Record("QualityGate", "Too many errors: " & Text.From(ErrorCount), null) else CleanRows</code> .
6	Add a separate diagnostic query that shows all error rows with their messages — a 'quarantine table' for manual review.
7	Discuss: this pattern implements a quality gate — tolerate small error counts, fail loudly when errors exceed a threshold.

RESULT

The nightly refresh no longer crashes on malformed data. Invalid rows are isolated with error messages. A quality gate raises an alert if errors exceed the acceptable threshold. A diagnostic query provides a quarantine table for investigation.

LAB 7

Control Flow and Advanced M

In this lab you apply all control flow and language mechanics — `let...in` structure, conditional logic, function definitions, error handling, and recursive functions — to build defensive, reusable M components.

Lab Scenario

Contoso Analytics needs a hardened, reusable ETL function library and a recursive hierarchy flattener for their organisational chart data. You will also add a multi-step validation pipeline and test it against deliberately malformed data.

NOTE

All exercises use M only — no ribbon clicks. Write all code in the Advanced Editor. Complete exercises in order — each builds on the previous.

Exercise 1: `let...in` and Conditional Logic (Estimated time: 20 minutes)

- Create a blank query named `ConditionalDemo`. Write a `let...in` with at least eight named steps: source data (a literal list of records), a date range check, a conditional grade assignment, a null-safe revenue calculation, and a final typed table.
- In `ConditionalDemo`, add a step that chains five conditions to assign a `SalesGrade` (Platinum / Gold / Silver / Bronze / Unqualified) based on Revenue thresholds from a config record.
- Replace the five `if/else` conditions with a lookup record pattern and `Record.FieldOrDefault`. Verify the output is identical.
- Add a conditional that tests whether a date column contains any nulls using `List.AnyTrue`, and raises a descriptive error if true.
- **STRETCH:** Implement a nested `let` inside `Table.AddColumn` to compute a multi-step `ShippingCost` with `Weight`, `Zone`, and `Express surcharge` as named intermediate values.

RESULT

`ConditionalDemo` produces a typed table with `SalesGrade`, `Revenue`, and `ShippingCost` columns. The lookup-record version is visually simpler than the chained `if/else` version and produces identical results.

Exercise 2: Function Definitions and Closures (Estimated time: 20 minutes)

- Create a blank query named `FunctionLibrary`. Its result should be a record of four functions: `FormatCurrency`, `SafeDate`, `IsWeekday`, and `NormaliseText`.
- `FormatCurrency(amount, optional decimals)`: formats a number as currency text. Use the `??` operator for the optional parameter with default 2.
- `SafeDate(v)`: returns `try Date.From(v)` otherwise null. Annotate with nullable date return type.
- `IsWeekday(d)`: returns true if the date is Monday–Friday. Use `Date.DayOfWeek`.
- `NormaliseText(t)`: trims, title-cases, and removes double spaces. Chain `Text.Trim`, `Text.Proper`, and a `Text.Replace` loop using `List.Accumulate`.
- Test each function against valid and invalid inputs. Verify `SafeDate` returns null (not error) for invalid date strings.

- **STRETCH:** Add a `MakeRounder(n)` factory function that returns a function rounding to `n` decimals. Demonstrate it is a closure by calling `MakeRounder(2)(3.14159)`.

RESULT

`FunctionLibrary` is a record of four named functions. Each function handles edge cases correctly. `MakeRounder` demonstrates closure capture of the `n` parameter.

Exercise 3: Error Handling and Defensive ETL (Estimated time: 25 minutes)

- Load the provided malformed CSV (it contains invalid dates, null keys, and negative revenue). Show that a direct `TransformColumnTypes` fails.
- Replace the failing step with row-level `try/otherwise` for the date column. Repeat for the revenue column. Verify the query completes with nulls for bad values.
- Add an `ErrorDetail` column using `'try (expr)'` (without `otherwise`). Capture `[HasError]` and `[Error][Message]` per row into a record-valued column. Expand it.
- Implement the multi-step validation pipeline from Pattern 2. Add at least four checks: has rows, no null keys, valid dates, non-negative revenue.
- Add a quality gate: raise a structured `Error.Record` if more than 5% of rows fail validation.
- Create a `Quarantine` query that shows only error rows with their `ErrorDetail` for investigation.
- **STRETCH:** Wrap the entire ETL query in a `try` block. On failure, return a one-row table with `Status='Error'` and a `Message` column instead of crashing.

RESULT

The ETL query tolerates individual bad rows, captures error details, enforces a quality gate, and provides a `Quarantine` table for investigation. The query never crashes on malformed input.

Exercise 4: Recursive Functions and Advanced Patterns (Estimated time: 25 minutes)

- Implement a recursive `Factorial` function using `@`. Test with inputs 0, 1, 5, 10.
- Implement a recursive `Greatest Common Divisor (GCD)` function: `GCD(a, b) = if b=0 then a else GCD(b, a mod b)`. Test with `GCD(48,18)=6`.
- Load `HierarchyData.json`. Implement a recursive `GetAncestorPath` function that builds a `'>'` separated ancestor chain for each employee. Include a depth limit of 15.
- Add the `hierarchy` function to `FunctionLibrary` from Exercise 2.
- Implement Pattern 3 (dynamic dispatch) with at least five formatters in a `Formatters` record. Test by loading a format type from a config record.
- Implement Pattern 4 (type-safe generic `SmartFormat`). Apply it to a mixed-type table using `List.Transform` on `Table.ColumnNames`.
- **STRETCH:** Implement a memoised recursive `Fibonacci` using `Table.Buffer` to cache already-computed values.

RESULT

`Factorial` and `GCD` work correctly. `Hierarchy flattener` produces correct ancestor paths with depth protection. `Dynamic dispatch` and `SmartFormat` apply correctly to mixed-type data. All functions are in `FunctionLibrary`.

Module Review and Takeaways

Review Questions

Is let...in a sequence of statements? How does M evaluate bindings?

Answer: No. let...in defines simultaneous name bindings, not sequential statements. M uses lazy evaluation — it only evaluates the bindings actually needed to produce the 'in' result. A binding that is never referenced is never evaluated. Order of evaluation is determined by dependency, not by the order bindings are written.

What is the difference between each _ and (x) => syntax?

Answer: each is syntactic shorthand for (_) => . Both define a single-parameter anonymous function. each _ is convenient for simple row/element expressions. Use explicit (x) => syntax when you need two parameters (accumulators), a named parameter for readability, or self-reference via @ for recursion.

What is the difference between try...otherwise and try (without otherwise)?

Answer: try...otherwise catches any error and returns the fallback value — you lose the error details. try without otherwise returns a record with HasError (logical) and either Value or Error fields. Use try...otherwise for simple safe conversions. Use try alone when you need to inspect, log, or branch on the error details.

Why is the @ operator needed for recursive functions?

Answer: Inside a let binding, the name being defined is not in scope within its own definition — so a function cannot call itself by name. The @ operator provides a special self-reference that makes the current binding available to itself, enabling recursion.

What is a closure and how is it used in M?

Answer: A closure is a function that captures variables from its enclosing scope. In M, any function defined inside a let...in automatically closes over all bindings visible at that point. This enables factory functions (MakeMultiplier), and means inner functions can access outer let bindings without needing them passed as parameters.

When should you use a lookup record instead of chained if/else?

Answer: Use a lookup record (Record.FieldOrDefault) when you are mapping keys to output values — especially when there are more than 3–4 cases, or when the mappings might change. Use if/else for logic that depends on comparisons, compound conditions, or null checks. Records are data; if/else is logic.

What is the purpose of Error.Record vs a plain text error message?

Answer: Error.Record produces a structured error with Reason, Message, and Detail fields. The Reason provides a machine-readable category; the Message is human-readable; the Detail can hold any record with diagnostic data. Power BI displays all three in its error dialogs. A plain text error message provides only the message — no category or detail.

What is dynamic dispatch and how is it implemented in M?

Answer: Dynamic dispatch selects and calls a function by name at runtime, without if/else chains. In M, store functions in a record (the dispatch table), then use Record.FieldOrDefault(Dispatchers, methodName, defaultFn) to select and call the right function. This is O(1), extensible without code changes, and eliminates long if/else chains.

Best Practices

- Name every `let...in` binding descriptively — the name should say what the step produces, not which function was called. Each name becomes an Applied Step: make it meaningful.
- Use nested `let` inside `Table.AddColumn` or complex function bodies to give names to intermediate values. Deeply nested one-liners are hard to read and debug.
- Always use `try...otherwise` for type conversions on user-supplied or API data. `Number.FromText`, `Date.From`, and similar functions throw on invalid input — always handle this.
- Use `try` (without `otherwise`) when you need to log or branch on error details. Use `try...otherwise` only when you truly want to discard the error.
- Always use `Error.Record` (not plain text) for production error messages. Structured errors are visible in Power BI error dialogs with full detail, making debugging tractable.
- Every recursive function must have a depth limit in its termination condition. Always test recursion with circular data. Without a limit, circular references crash the refresh.
- Store reusable functions in a record in a single staging query. All other queries reference it by name. This is the only scalable pattern for shared M logic in Power BI.
- Use the `??` null-coalescing operator rather than `'if x = null then default else x'`. It is shorter, clearer, and handles the null case correctly in all type contexts.
- Use the `Record.Combine defaults-and-overrides` pattern for all optional function parameters. This is more extensible and cleaner than chains of `if-null` checks.
- Add quality gates: validate schema, row count, and key integrity before loading to the model. A failed quality gate should raise a structured error — never silently load bad data.

Common Issues and Solutions

Circular dependency error in `let...in`

Cause: A binding references another binding that, directly or indirectly, references it back. M cannot resolve circular dependencies.

Solution: Restructure the bindings so the dependency graph is acyclic. If genuine circular dependency is required (rare), use `Record.AddField` with `delayed=true`.

Stack overflow in recursive function

Cause: The recursion is too deep — either the data is deeper than expected, or a circular reference causes infinite recursion.

Solution: Always add a depth limit: `if depth > 100 then error "Max depth" else @recurse(...)`. For deep iteration (>100 levels), replace recursion with `List.Generate`.

`try...otherwise` swallows errors that should fail

Cause: You used `try...otherwise` too broadly and a genuine data problem (not a format issue) is being silently converted to null.

Solution: Be specific about what you wrap in `try`. Wrap only the one expression that may legitimately fail (e.g. `Date.From`). Do not wrap entire multi-step expressions in a single `try`.

each _ refers to the wrong thing inside nested functions

Cause: You have an `each` inside another `each` and `_` refers to the inner context, not the outer row.

Solution: Give the outer parameter an explicit name: (row) => ... List.Transform(someList, each _ + row[X]). Nested each _ always refers to the innermost each context.

Optional parameter receives null when caller omits it

Cause: M does not distinguish between 'omitted' and null for optional parameters. Both appear as null inside the function.

Solution: Always use ?? to provide a default: decimals ?? 2. Or use the Record.Combine options pattern, where an empty record {} is passed for omitted options.

Key Constructs — Module 7 Quick Reference

All language constructs and functions introduced in this module.

Construct / Function	Category	Key Behaviour
<code>let Name=expr, ... in expr</code>	Structure	Define named bindings; return 'in' expression
<code>if A then X else Y</code>	Control Flow	Conditional expression — always produces a value
<code>each</code>	Function	Shorthand: <code>() => ; _</code> is current row/element
<code>(p) => expr</code>	Function	Explicit anonymous function; named parameter p
<code>(p as T) as R => expr</code>	Function	Typed function; T=param type, R=return type
<code>@FuncName(args)</code>	Recursion	Self-reference in let binding; enables recursion
<code>try expr otherwise fallback</code>	Error	Safe eval; returns fallback if expr errors
<code>try expr</code>	Error	Returns <code>[HasError, Value]</code> or <code>[HasError, Error]</code> record
<code>error "message"</code>	Error	Raise a plain text error; stops evaluation
<code>error Error.Record(r,m,d)</code>	Error	Raise structured error with Reason/Message/Detail
<code>x ?? default</code>	Operator	Null-coalescing: return default if x is null
<code>Value.Is(v, type T)</code>	Type	true if value v conforms to type T
<code>Value.Type(v)</code>	Type	Return the M type of value v
<code>Type.Is(t1,t2)</code>	Type	Check if type t1 is compatible with type t2
<code>List.AllTrue(list)</code>	Validation	true if all logical values in list are true
<code>List.AnyTrue(list)</code>	Validation	true if any logical value in list is true
<code>Error.Record(r,m,d)</code>	Error	Build a structured error record
<code>Record.FieldOrDefault(r,f,d)</code>	Dispatch	Safe field lookup — enables dispatch tables

Module Summary

- `let...in` defines simultaneous name bindings, not sequential statements. M is lazily evaluated — only bindings needed for the 'in' result are evaluated. Name every binding descriptively; each name becomes an Applied Step.
- `if/then/else` is a conditional expression, not a statement — it always produces a value and both branches are required. Chain conditions with nested `else if`. For value mapping, prefer lookup records over long `if/else` chains.
- `each` is syntactic sugar for `(_) => .` Use explicit `(x) =>` syntax for two-parameter functions, named parameters for readability, and recursive self-reference.
- Functions are first-class values in M — they can be stored in variables, records, and lists, and passed as arguments. This enables dynamic dispatch, factory functions, and closures.
- `try...otherwise` safely converts errors to fallback values. `try` alone returns a record with `HasError` and `Value/Error` fields, enabling error inspection and logging. Always use `Error.Record` for structured, debuggable errors.
- The `@` operator enables recursion by providing a self-reference inside a `let` binding. Always include a depth limit. For deep iteration, prefer `List.Generate`.
- Five advanced patterns: ETL function library, multi-step validation pipeline, dynamic dispatch, memoisation, and type-safe generic functions — all enabled by M's first-class function support.

NOTE

Next Module — Module 8: Custom Connectors and API Integration. Covers `Web.Contents`, `Json.Document`, paginated REST APIs, OAuth, query parameters, and building a production-grade API connector using the full M language toolkit from Modules 1–7.